

Lab1 DataLab 实验报告

——游舒恬 25803050105

实现每个函数的思路

P1 signMask 直接将1左移 31 位得到。

P2 bitXor 先翻译异或的定义： $x \oplus y = (x \& \sim y) \mid (\sim x \& y)$ ，再利用德摩根定律，将 $\mid$ 转为 $\sim(\sim a \& \sim b)$ 。

P3 negativePart 用 $x \gg 31$ 得到全1（负数）或全0（非负），与 $\sim x + 1$ （即 $-x$ ）按位与，即可构造出题目所需返回值。

P4 copyByteWithin 用 $(0xFF \& (x \gg (src \ll 3)))$ 取出源字节，左移到目标位置，再用掩码清除目标字节原值，最后按位或合并。

P5 logicalShift 先做算术右移，再构造掩码清除高 n 位，得到逻辑右移。

P6 swapNibblePairs 构造掩码取出所有高4位，右移4位；取出所有低4位，左移4位。最后合并。

P7 secondLowestZeroBit 先通过二进制数加1会将最低0位变为1的特性，通过构造将最低0位改为1，其他位不变；再求新 x 的最低0位，即为原数第二个最低0位。

P8 oddParity 将高半部分与低半部分逐步计算异或： $x \oplus= x \gg 16$ ； $x \oplus= x \gg 8$ ；...，最终最低位即为所有位的异或和。由于奇数个1时异或和为1，根据题目要求，结果需要再进行一次取反。

P9 rotateRightBits 循环右移，先得到 n 除以32的余数，才是真正的右移位数。然后计算左移部分 $x \ll ((\sim n + 1) \& 31)$ 和右移部分 $(x \gg n) \& \sim((\sim 0) \ll ((\sim n + 1) \& 31))$ ，最后合并。

P10 roundEvenPow2 舍入到最近的 2^n 倍数，中点取偶数商。先做常规四舍五入，若余数恰好为中间值且原商（四舍五入前）为偶数，则商减1。最后左移还原到原位数。将代码中重复使用的几个变量提取到最开头： $pow2n = 1 \ll n$ ， $half = pow2n \gg 1$ ， $mask = pow2n - 1$ 。

P11 midpointTowardFirst 先判断符号差和大小关系，计算和并处理溢出（由于两个有符号数相加，通过符号位即可判断并处理溢出）。直接将和整除2，若和为奇数（即中点有两个值），根据 x 是否大于 y 决定是否将结果加1。

P12 isBetweenEitherOrder 先判断 x 是否等于端点，是则直接返回1。否则，判断 $(a < x)$ 与 $(x < b)$ 是否同真同假，相等则 x 在区间内。由于是两个有符号数比较大小的关系，直接用减法和符号位即可判断。

P13 mul5Sat 由于实验限制字面量只能取0-255，先通过移位和或运算构造出正负溢出的阈值，并用 $\text{sign_x} = x \gg 31$ 得到 x 的符号扩展。正溢出的判断条件是 x 非负且大于正阈值；负溢出的判断条件是 x 为负且小于等于负阈值。根据两个条件构造出各自的条件判断掩码，再合并得到总溢出掩码 $\text{ovf} = \text{pos_ovf} \mid \text{neg_ovf}$ 。饱和值根据符号选择，负数取 INT_MIN ，非负数取 INT_MAX 。计算未溢出时的乘积 $\text{mul5} = (x \ll 2) + x$ 。最后根据溢出掩码选择输出：若溢出则输出对应的饱和值，否则输出 mul5 。

P14 classifyAdd3 判断 $x + y + z$ 是否超出 int 范围。模拟64位加法：低32位相加，计算进位；高32位即为各数符号扩展之和加进位。如果精确和能用32位有符号表示，那么它的高32位必须恰好等于低32位的符号扩展，即 $\text{sum_high} == \text{sign_low}$ 。因此先比较高32位与低32位的符号扩展，接着用减法和符号位判断两者大小关系，得到三个掩码，对应表示三种大小关系。当 $\text{sum_high} < \text{sign_low}$ 时，精确和小于 INT_MIN ，返回-1；当 $\text{sum_high} > \text{sign_low}$ 时，精确和大于 INT_MAX ，返回1；当 $\text{sum_high} == \text{sign_low}$ 时，返回0。

P15 floatScaleThreeHalves 浮点数题目先拆解符号、阶码、尾数部分，并按照非规格化、规格化、特殊值讨论。特殊值直接返回原值。非规格化： $M = \text{frac}$, $E = -149$ ；规格化： $M = (1 \ll 23) \mid \text{frac}$, $E = \text{exp} - 150$ 。乘以二分之三转化成 $M *= 3$, $E -= 1$ 。若原为非规格化，则除了舍入外还需要考虑数值超过非规格化表示范围的特殊情况（若结果 $\geq 2^{23}$ 转为规格化）。若原为规格化，找 M 最高位 k （只可能是24或25），计算 $\text{exp_new} = k + E + 127$ ，右移舍入，拼合结果返回。

P16 floatRoundEven 同样先拆解符号、阶码、尾数。特殊值原样返回。非规格化： $M = \text{frac}$, $E = -126$ ；规格化： $M = (1 \ll 23) \mid \text{frac}$, $E = \text{exp} - 127$ 。若 $E \geq 23$ 已是整数直接返回。否则分情况讨论：若 $E < 0$ ，根据 $E == -1$ 和 M 决定舍入到1或0。否则分离整数和小数部分，按round-to-nearest-even舍入，再将整数转回浮点。

注：P15将 M 视为一个整数便于计算，因此 E 需要补偿 M 作为整数时小数点被右移的23位，规格化数取 $E = \text{exp} - 150$ 。P16中将 E 视为实际指数直接反映数值的量级，便于判断小数点是否已移出全部小数位，因此 $E = \text{exp} - 127$ 。

P17 float_i2f 先处理特殊值。然后取绝对值，找最高位 k ， $\text{exp} = k + 127$ 。根据位数讨论：若 $k > 23$ ，右移 $k - 23$ 位并做舍入，舍入后需要判断是否导致进位，若此时 $\text{frac} \geq 2^{24}$ 则右移一位并 $\text{exp}++$ 。若 $k \leq 23$ 直接左移对齐。拼合符号、阶码、尾数得到返回值。

P18 bitCount 构造掩码，分治并行加法：先两两相加，再四四、八八、十六、三十二合并，最终得到总的1个数。

P19 bitReverse 构造掩码，分治交换：先相邻位交换，再2位、4位、8位、16位块交换，最后合并，得到32位交换。

以上内容均经过自己的思考，部分题目使用了ai获取一些思路提示或关于简化操作数的方法提示，但都没有让ai直接给出完整实现或直接生成代码。

实验结果

在终端中执行 `./check_ops.py bits.c` 后的截图：

```
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 16/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 2/16 operators
roundEvenPow2: 17/24 operators
midpointTowardFirst: 31/32 operators
isBetweenEitherOrder: 28/48 operators
mul5Sat: 8/30 operators
classifyAdd3: 35/52 operators
floatScaleThreeHalves: 48/60 operators
floatRoundEven: 35/65 operators
float_i2f: 32/40 operators
bitCount: 30/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

(rotateRightBits 应该是14个操作数，不知道为什么测出来是2)

在终端中执行 `./btest` 后的截图：

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse
Total points: 110/110			